Graphical Abstract

Distilling the Complexity of Agent-Based Simulations into Textual Explanations via Large Language Models

Noé Y. Flandre[a,b], Philippe J. Giabbanelli[b]*

Highlights

Distilling the Complexity of Agent-Based Simulations into Textual Explanations via Large Language Models

Noé Y. Flandre[a,b], Philippe J. Giabbanelli[b]*

- Generated summaries of ABM simulations using LLMs and two summarization methods
- Evaluated reports with BLEU, METEOR, ROUGE, and reading ease metrics
- Prompt elements improved readability but had minimal effect on overlap metrics
- Factorial analysis revealed key interactions between LLMs, prompts, and metrics
- Simulation models can be transformed into executive summaries for non-modelers

Distilling the Complexity of Agent-Based Simulations into Textual Explanations via Large Language Models

Noé Y. Flandre[a,b], Philippe J. Giabbanelli[b]*

[a]*IMT Mines Alès, 6 Avenue de Clavières, Alès, 30100, Occitanie, France*
[b]*Old Dominion University, 5115 Hampton Blvd, Norfolk, 23529, VA, USA*

Abstract

Communicating the design and results of Agent-Based Models (ABMs) subject matter experts and other stakeholders can be challenging, which hinders participation and limits trust in the model. Advances in Generative AI make it possible to create textual summaries to communicate ABMs, thus complementing works on statistical and visualization techniques. While prior work used Large-Language Models to translate conceptual models into text, we extend this to simulation models such as ABMs. We propose a fully automated pipeline that extracts contextual information from ABMs in Netlogo and performs simulations, then summarizes them using state-of-the-art LLMs along with abstractive and extractive summarization algorithms. Our optimization considers three LLMs (GPT-4o, Claude 3.5 Sonnet, DeepSeek) and different prompt elements (setting a role, including an example, generating insights), which are analyzed via a factorial design of experiments over ABMs previously published by various research groups. Our analysis reveals that the summarization algorithm is the dominant driver of content-based metrics (BLEU, METEOR, ROUGE), explaining up to 95% of variance, while prompt elements such as insights or example mainly influence the readability of the report. Our work demonstrates the feasibility of generating executive summaries from simulation models and exemplifies the benefits of using the well-established techniques of designed experiments from the simulation literature to evaluate the interaction of prompt elements and LLMs. While our method can contribute to making simulation insights more accessible to decision-makers and stakeholders beyond the modeling community, our methods are currently limited to ABMs with sufficient documentation to

Email address: *pgiabban@odu.edu ()

provide a context.

Keywords: Agent-Based Modeling, Automatic Summarization, Factorial Analysis, Large Language Models, Simulation Communication, Simulation Output Interpretation

---

1. Introduction

Explainability and transparency are important considerations in Modeling & Simulation (M&S) as they build trust and confidence in the validity of models, thus supporting the translation from model's insights into practice. The creation of understandable narratives to accompany models is particularly beneficial for end-users who may not have the technical expertise to interpret raw simulation data directly or appreciate the limitations of specific model-building processes [1]. Explainability and transparency are multi-faceted, as they can be done at the local (e.g., sensitivity to changes in individual parameters [2]) or global level (e.g., comparing point-estimates from a simulation with ground truth), and they cover the production of a model [3] (how was it created?) as well as the intended use of the final product (Figure 1). When conceptual models strive to be comprehensive, they are at risk of becoming "spaghetti diagrams" [4], thus turning into unintended illustrations of complexity that offer little support for global explanations of the model and hinder translations into practices. The Foresight Obesity Map is a case in point, as it was depicted by researchers as an "almost incomprehensible web of interconnectedness [in which] the scale and number of interactions make it difficult to see how one might use it in any practical way to develop systemic approaches" [5], with others stating that the model "looks more like a spilled plate of spaghetti than anything of use to policymakers" [6]. The solution is not necessarily to force a model into a reduced scope or to omit mediating effects and multiple consequences [7], as oversimplified models may no longer be fit for purpose [8]. For example, public health models require the integration of biomedical, social, and environmental variables [9] so these models often need to be comprehensive and clearly reflect unintended consequences in order to serve their purpose, even if that initially appears to come at the expense of explainability and transparency.

Given the necessity of keeping models fit-for-purpose while addressing explainability and transparency, a possible solution could be that modelers provide all necessary descriptions. However, analyses of descriptions surround-

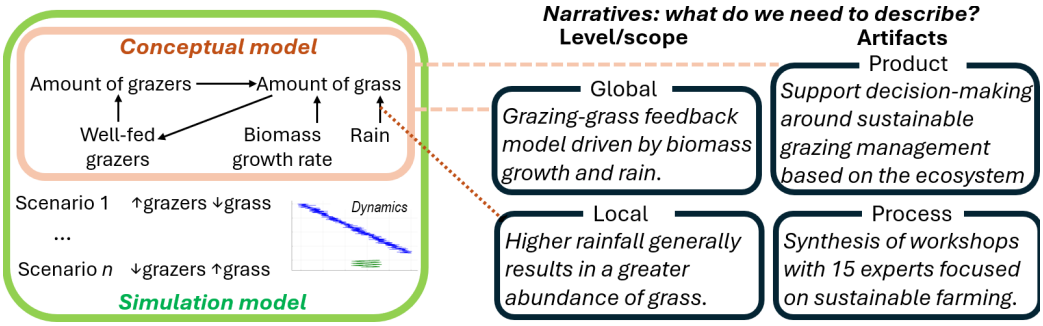

Figure 1: Several works have used GenAI (particularly OpenAI's GPT) to translate conceptual models to text, often with a focus on local elements [10, 11] and occasionally by using summarization techniques to provide a sense of the entire model. In this paper, we generate text for simulation models, which adds dynamic scenarios.

ing conceptual models and their production repeatedly showed that modelers do not systematically describe the model development process [3, 12] or the sources of model elements such as relationships between variables [3]. The realization that the burden cannot solely be placed on modelers to provide more descriptions has resulted in a renewed interest for automation via Natural Language Generation. This is not merely a matter of generating (any) text, as simple templates could produce strings that describe a conceptual model such as "A increases B. B decreases C." Advances in cognitive sciences show that certain representations of events and causality support language processing tasks in readers [13], thus we seek textual descriptions that allow readers to form correct views about the model being described. Generative Artificial Intelligence (GenAI) solutions such as GPT have been used to translate large conceptual models into descriptions at the local level (e.g., model components such as relationships [10, 11]) or at the global level (e.g., decomposing a model into blocks [14] or summarizing its comparison with ground-truth data [15]) that have satisfactory readability levels and remain factually correct. This flurry of research on model-to-text complements the significant research growth on text-to-model [16, 17, 18, 19].

As significant advances have been made on automatically producing narratives for conceptual models by using GenAI, the next frontier is to create descriptions for simulation models. The simulation-to-text task is more challenging than a model-to-text translation because simulations are dynamic systems, thus we have to convey both the rationale and structure of a model (as in model-to-text) and its dynamics in response to various stimuli or in-

put scenarios (Figure 1). To date, there is a paucity of works in this domain. Notable exceptions include the study of Verduzco and colleagues who used GPT-4 to describe input files for the Large-scale Atomic/Molecular Massively Parallel Simulator (LAMMPS), thus demonstrating the feasibility of employing GenAI to address lacking descriptions of simulation methods [20]. While GenAI can create narrations by interacting with agents or simulation logs [21], its primary goal was to generate compelling stories for the game industry (e.g., the hero, warrior, wizard, and dragon from Aoki *et al.* [22]) rather than to increase the explainability or transparency of a simulation serving as a decision-support tool. There is thus a demonstrated need to automatically produce descriptions that alleviate, at least partly, the challenges of conveying simulation models to a target audience.

In this paper, we focus on translating Agent-Based Models (ABMs) into text. ABMs simulate interactions between individual entities ('agents') and their environment, with successful applications across many domains including ecological and socio-ecological systems [23, 24], sociology [25], or political decision making [26]. While ABMs are powerful tools to explore complex systems, their results are notoriously challenging to communicate due to the difficulties in analyzing and interpreting the multidimensional output data [27]. This difficulty is exacerbated when trying to communicate findings [28], particularly to non-experts who need to make informed decisions based on these outcomes [29]. Furthermore, the amount of data generated can be significant for detailed simulation models or those with large search spaces, but readers have a limited cognitive capacity. In other words, trying to solve the challenge of explainability by translating every aspect of a simulation model into an enormous phone book would just displace the problem. ABMs thus require distilling information into concise, yet comprehensive insights, requiring sophisticated analytical approaches [27].

Our main contribution is to generate concise reports from ABM simulations, at a level of readability that is appropriate for non-modeling experts [30]. This overarching aim is realized as follows:

- We design, implement, and evaluate a method that uses GenAI to create short (executive) reports of simulation models that describe their parameters, purpose, and the main trends from simulated scenarios.
- We leverage Large Language Models equipped with multimodal learning (GPT-4o, Claude Sonnet) such that dynamic trends are automatically identified from the time series generated by simulations.

- Our empirical evaluation uses multiple Agent-Based Models and various Large Language Models, demonstrating strong qualitative and quantitative scores that indicate satisfactory levels of readability and factuality.

The remainder of this paper is organized as follows. In Section 2, we provide background information to keep the manuscript self-contained, with special emphasis on techniques from Generative AI and their intersection with simulations. Building the technical foundations summarized in our background, we present our methodology in Section 3, including the processes of data extraction, report generation, and our evaluation methods. Our results are presented in Section 4, followed by a discussion of our findings in Section 5. Finally, we provide concluding remarks in Section 6. To support transparency and replicability, our work is provided open-source on a third-party repository at `TBD!`.

2. Background

Our paper is motivated by the realization that simulation models such as ABMs can be complex and produce large amounts of data, which need to be succinctly conveyed to a variety of end-users (subsection 2.1). We propose using Large Language Models (LLMs), which have already been employed in several simulation tasks (subsection 2.2). We focus on using LLMs to generate summaries, which can be evaluated both qualitatively and quantitatively (subsection 2.3) in response to different prompt design techniques (subsection 2.4).

2.1. Conveying Agent-Based Models to end-users

ABMs can capture interactions between *multiple agents* (characterized by a number of dynamic attributes), over *several steps* (or 'ticks'), through a *variety of scenarios* that can be performed *multiple times* to account for stochasticity. The sheer volume and complexity of the data produced by ABMs can overwhelm users, making it difficult to extract actionable insights directly from the raw outputs [30]. In addition, the design of the ABM and simulation can become complex, as it involves numerous agent attributes, several data- and theory-driven rules, along with various scenarios consisting of combinations of parameter values. This information can be comprehensively captured through standardized approaches to describe ABMs, such as

ODD [31] and its extensions [32, 33], and some aspects can be covered through a UML diagram (emphasizing software design) or reporting standards such as RAT-RS for data use in ABMs specifically [34] or the STRESS guidelines for simulations in general [35]. However, these standards and guidelines are not systematically used by the modeling community [36] and they can be challenging for stakeholders and model commissioners, who are not always modelers but need to get a sense of what the model 'does' and how to use it without parsing large volumes of technical information.

Several works have sought to improve communication for the design of ABMs and/or their simulation results, particularly for non-technical audiences. For example, Hall and Virrantaus used diagrams to convey the rules of ABMs [37], which was more recently echoed by Szangolies and colleagues in creating a visual version of the standardized ODD [38]. Visualizations have been an important tool and research area to communicate ABM for over a decade, across all phases of the modeling process [39], for instance using the popular `NetLogo` software [40]. While statistical techniques are commonplace in simulations (e.g., choosing a number of repeats for a target confidence interval), they often support modelers [41]. On occasions, statistics have been employed to convey ABMs to practitioners beyond M&S such as statisticians [42] or a broader audience [27]. As shown in the next subsection, advances in GenAI have provided new opportunities to explain ABMs as text, thus complementing studies on visualizations and statistics.

2.2. LLMs and Simulation

LLMs have intersected with simulations in several ways. While only one task (simulation-to-text) is directly relevant to the present work, we briefly discuss the names and techniques involved in three related areas to avoid confusion. First, in *generative Agent-Based Modeling* or 'LLM-augmented social simulations' [43], the decision-making modules of the agents are replaced or extended with LLMs (Figure 2). This allows agents to react to messages and to explain their reasoning [44, 45], and it also transforms the challenge of crafting behavioral rules onto a prompt engineering task that accounts for the agents' attributes [46]. As pointed out by Larooij and Tornberg, "generative ABMs are still in a stage of early experimentation" [47], with prototypes emerging in late 2024 and early 2025. All ABMs used to generate simulations in our paper are 'traditional' as they use sets of rules to express behavior rather than LLMs. Second, in *Agentic AI*, the emphasis is on using a set of specialized agents to interpret a textual request [48]. For example,

"approach potential clients" can trigger a market discovery agent that looks for suitable companies, then another agent extracts email addresses, and a third agent crafts tailored emails. In other words, agentic AI decomposes a problem so that it can be solved by a set of specialized, autonomous entities. The goals of the traditional ABMs studied in this paper are different, since they provide simulations as decision-support tools so that end-users can estimate the effects of scenarios defined through input parameters. Third, there is a wealth of studies on *generating simulations from text* (Figure 2), drawing from earlier works on extracting ABM concepts from narratives [49, 50], then using questions and answering systems to automate the extraction [51, 52] and finally generating simulation code in a target ABM platform such as `NetLogo` [53, 54]. In this paper, we study the reciprocal transformation: instead of building an ABM from text, we use existing ABMs to generate a textual description.

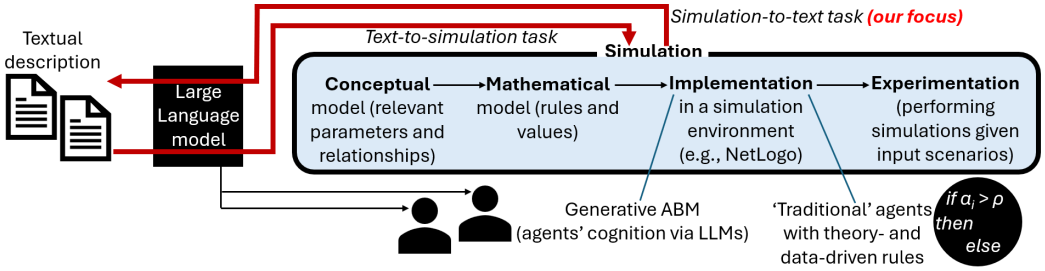

Figure 2: LLMs have been used to support several aspects of simulations, particularly to generate simulations from text or to replace the agents' decision-making module. In this paper, we focus on simulation-to-text.

As tools such as GPT make it possible to translate simulations into text, the idea of using GPT to explain simulations was proposed [15, 55] and realized in a few studies. In particular, Jackson and Rolf noted the technical complexity of simulations and translated them into natural language for domain experts, by translating the Python source of inventory management systems into English. While the results use a repetitive style that does not match the readability level expected of a report, this proof-of-concept demonstrated that GenAI could know enough about a simulation model in order to depict it with factually correct sentences [56]. Gürcan also noted that agents equipped with LLMs (i.e., generative ABM) could help with explainability, as the agents "can generate natural language explanations for their actions, decisions, and the underlying mechanics of the simulation" [43].

Note that both approaches can generate a large amount of low-level information, either by translating the whole simulation code to text [56] or by asking each agent to explain how it decided and acted [43]. There is thus a need to distill the information, which we accomplish in this paper by using summarization techniques as presented in the next subsection.

2.3. Evaluating LLM-generated summaries

When the output from LLMs is too verbose for practical use, we can further request that the LLM summarizes it. Summarization algorithms fall into two broad categories: *extractive* (copy-paste the most relevant parts of the original text) or *abstractive* (synthesizing information by paraphrasing the text). Extractive summarization prioritizes factual content and offers computational efficiency, making it well-suited for summarizing informative texts, as illustrated by the well-known BERT [57] (Bidirectional Encoder Representations from Transformers) algorithm. In contrast, abstractive summarization techniques aim to capture the essential meaning using novel phrasings, as seen with BART [58] (Bidirectional and Autoregressive Transformers for Pre-training).

Since summarizing is about communicating, it is important to identify the audience and the format [59]: do we seek one paragraph for an email to a modeler, a five minutes elevator pitch to a potential model commissioner, or a written overview for subject-matter experts? *Executive summaries* are a common format, found across application areas from military simulations [60] to behavioral modeling [61]. Executive summaries typically require concise and focused content yet powerful for the readers to take action [62], often coming down to 1 or 2 pages with no more than 1000 words [63].

Evaluating the results produced by LLMs involves both quantitative (our focus) and qualitative methods (described in supplementary section S1). To effectively evaluate LLMs and avoid common pitfalls, it is essential to develop a comprehensive suite of metrics rather than relying on a single metric [64]. *Quantitative evaluation* often use metrics such as *BLEU*, *METEOR*, and *ROUGE*, which are standard in summarization tasks, to compare the generated content against a ground truth [65, 66]. BLEU (Bilingual Evaluation Understudy) measures the precision of n-grams in the generated text compared to a reference text, using a scale from 0 to 1, where higher scores indicate better quality. Prior studies showed performances reaching 24.58% for this metric on summaries from BART [67] and 63% from BERT [68] . METEOR (Metric for Evaluation of Translation with Explicit ORdering) evalu-

ates translation quality based on precision, recall, and alignment, producing scores ranging from 0 to 1, with higher scores representing better quality. Researchers have reported results on summaries using BART (73.72%) [67] along with BERT (33%) [69]. ROUGE (Recall-Oriented Understudy for Gisting Evaluation) measures recall by comparing the overlap of n-grams between the generated text and reference texts. It includes several variants, such as ROUGE-N (measuring n-gram overlap), ROUGE-L (measuring longest common subsequence), and ROUGE-W (measuring weighted longest common subsequence), with scores typically ranging from 0 to 1, where higher scores indicate better performance. Prior works display results for ROUGE-1 that are approximately of 26.60 for BERT summaries and of 52.53 for BART summaries [70], for ROUGE-2 results approach 11.11 for BERT summaries and 21.83 for BART summaries [70], and lastly for ROUGE-L we have 24.29 for BERT and 49.75 for BART [70]. In addition, *the Flesch Readability Score*[71] assesses the ease of reading a text based on sentence length and word syllable count, with scores ranging from 0 to 100. Higher scores indicate easier readability, with 90-100 being easily understandable by an average 11-year-old student, and 0-30 being best understood by university graduates. Previous studies on LLM have shown a Flesch Reading Score between 24.88 and 58.01 [72].

2.4. Prompt engineering and experimental design

Effective prompt engineering is critical for leveraging the capabilities of LLMs in generating high-quality explanations [73]. Rather than relying on random attempts, prompts must be carefully designed to include essential elements such as roles, examples, and insights. This structured approach ensures that the generated content is relevant and informative [74]. The design of LLMs can be articulated along several dimensions [75], which can be binary (e.g., whether to defining a role and style) or multi-valued (e.g., whether the goal is expressed as looking-up information or creating). A notable challenge is that having more prompt elements does not guarantee better results [76]. Intuitively, 'throwing the kitchen sink' at the LLM may confuse it rather than guide it. Since we do not know *a priori* which elements to include, several prompts can be designed and their effects evaluated empirically.

The M&S studies that used prompt engineering have used several strategies to design their experiments. Several works have used the experimental design of varying *one factor at a time* [77, 78, 79], which involves systematically changing a single variable while keeping all other factors constant to

isolate its individual effect on the outcome. This simple design avoids the complexity and costs of testing all possible combinations of multiple factors, at the expense of missing how prompt choices influence each other. Other studies have used a 2^k factorial design with k binary prompt elements, which allows researchers to study both individual effects and interactions between factors efficiently at the expense of generating more prompts. For example, Zhao and colleagues tested a flat vs. hierarchical word representation (one binary factor) and whether to use a zero-shot approach that does not give information or to provide metaproperties documentation (another binary factor), resulting in a 2^2 design [80]. To date, no study has used a $2^k r$ factorial design, that is, an experimental setup that *also accounts for randomness* in the output by performing r repeats of each experiment.

In a 2^k experiment, the analysis involves decomposing the total variance in the outcome (e.g., the BLEU or ROUGE scores) into contributions from main effects (1st-order, individual factors) and interactions (2nd-order for pairs of factors, higher-order for combinations). For example, in the 2^2 design above, results can be decomposed into the individual effect of word representation and documentation, and their joint effect. Although the use of a factorial design is scarce to evaluate prompt elements in GenAI, this design of experiments is very well established in the simulation literature [81, 82].

3. Methods

3.1. Overview

Our methods are divided into three main parts (Figure 3). First, we obtain data about the ABM (subsection 3.2). Given a set of parameters, we run several simulations to obtain results from the model. We also extract data from the simulation file: we parse the documentation, the Graphical User Interface (GUI), and the comments from the code. Then we generate a report using this information (subsection 3.3). We transform simulation results into images, thus leveraging the multimodal learning ability of the LLMs (GPT-4o and Claude 3.5) to generate both a context and a trend analysis. A summarization algorithm reduces the number of paragraphs to the length suitable for an executive report. We optimize the prompt engineering process using a systematic Design of Experiments (DoE) to ensure the generated reports are both accurate and informative. Finally we evaluate the report (subsection 3.4) using qualitative and quantitative methods, performed both by human readers and by LLMs.

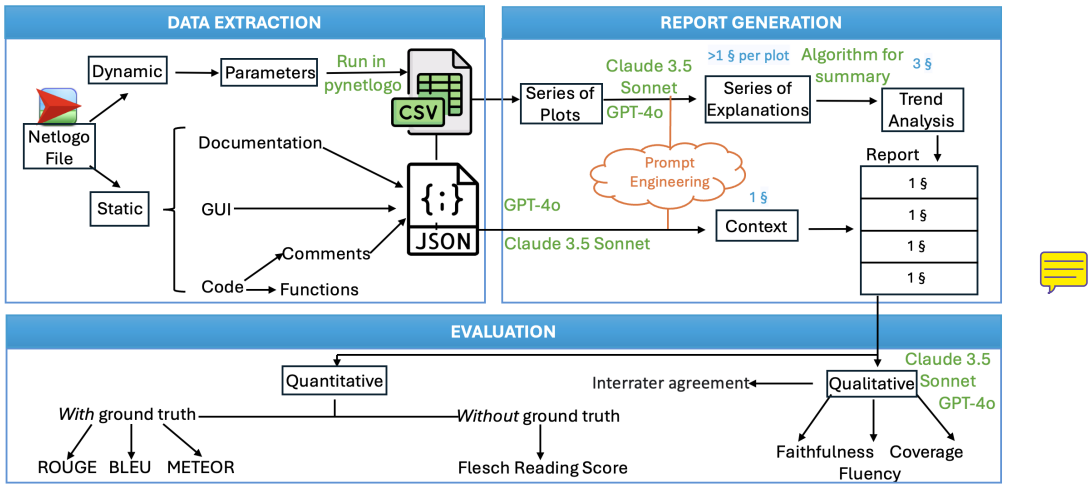

Figure 3: Our procedure consists in three main parts. Each part is explained in turn, in a dedicated subsection I NEED TO REDO THIS AT THE END

3.2. Data Extraction

An ABM that adheres to professional standards may include comments in the code, some documentation (e.g., at least a README file), and a minimal visual interface (i.e., the model is not solely driven in command line). This provides potentially valuable information for a model as it can explain its purpose or give context to its parameters. We thus extract this *static* data in addition to running the model to obtain *dynamic* simulation results. While our implementation uses ABMs coded in `NetLogo`, the automatic extraction process described below can be modified to accommodate other platforms.

For the *static data*, we extracted three components (Figure 4). First, we parse the *documentation*, ensuring that we ignore any default content (e.g., default headers from NetLogo) so that the information extracted was written by modelers specifically for their model. We also ignore URLs, as they may lead LLMs to retrieve information from webpages that create hallucinations. Second, we parse the *GUI* to identify modifiable elements and their range of values (e.g., sliders) as well as default values. These settings tell us how to interact with the model to generate simulations. Third, we isolate *comments* from the code using regular expressions. The extracted static data provides essential context for the ABM and we organize it as a JSON file, since the hierarchical format can organize diverse elements like documentation, code comments, and GUI configurations.

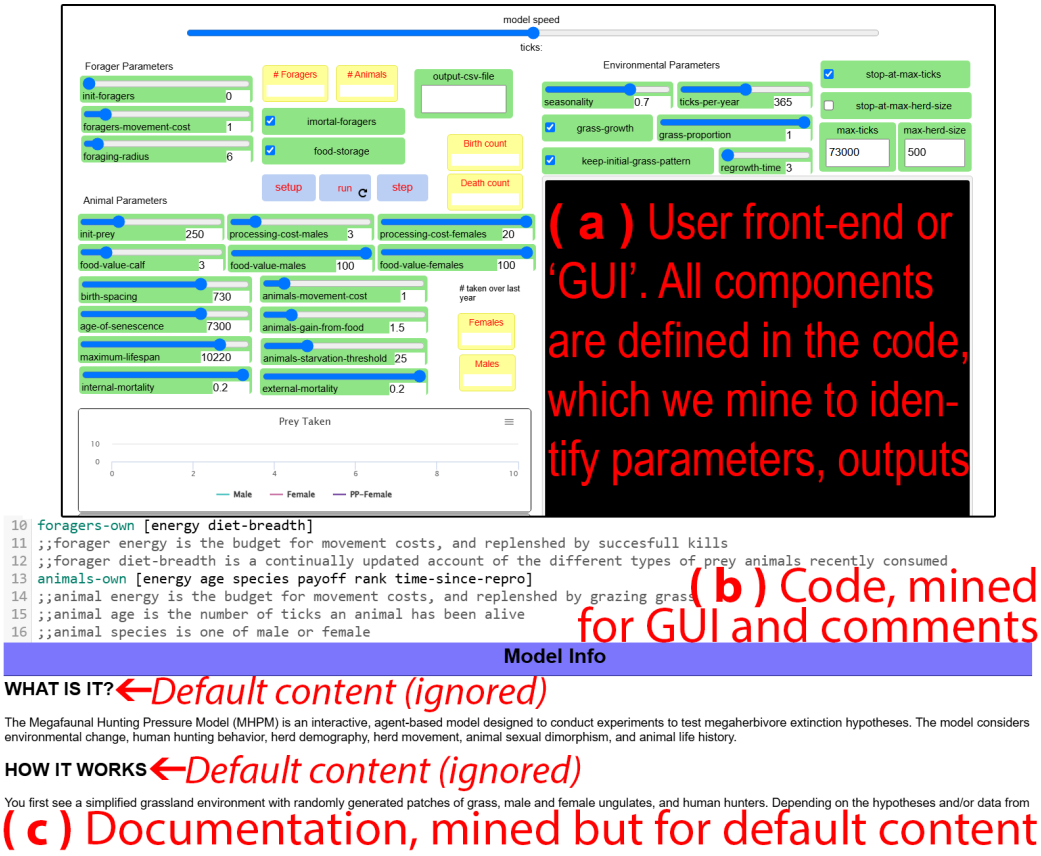

Figure 4: A NetLogo model is formed of three tabs: a front-end (a), the code that we mine (b), and a documentation that we also mine (c). The code contains both the model logic ('back end') and the visuals ('front end') such as the location of sliders, their range and default values. This example is from the Megafaunal Hunting Pressure Model, used in our experiments (Section 4.1).

The *dynamic data* consists of simulations that we run. Thanks to information provided from the model, we identify initial parameter values, the metrics to follow through the experiment, and the halting condition. For example a parameter is the 'number of agents: 1000', a metric is to 'track the average quantity of milk consumed by the agents', and a halting condition is to 'run for 73000 time steps'. We run simulations by piloting a model externally (i.e., from Python) and we save the results into a CSV file as it provides a standardized format for storing and analyzing structured data.

3.3. Report Generation by Prompt Engineering

The report is generated in three steps. First, we generate the *context* (Prompt 1): purpose of the model, key parameters, and experimental conditions. Second, we describe each of the simulation runs via Prompt 2, that is, the prompt is called repeatedly to transform each datasets into a narrative. Note that these two parts do not have a one-to-one mapping with the static and dynamic features discussed in the previous subsection: the context tells readers about the parameters (static) and how they've been set for simulations (dynamic), while the results (dynamic) are interpreted in light of the model's structure and aims (static). Third, the descriptions produced by the two prompts are unrestricted in lengths, but an executive summary needs to be concise. We thus perform post-processing by considering both abstractive and extractive summarization, along with cleaning steps for punctuations and other symbols. The three parts are detailed in this section.

3.3.1. Prompts and 2^3 factorial design

The prompts can be formulated in several ways and the best options are not all known, thus they need to be assessed experimentally (see subsection 2.4). To this end, we optimized the prompt engineering by using a systematic Design of Experiments known as 2^k factorial design, where k is the number of features considered in the prompt. Our approach uses $k = 3$ features detailed below (role, style, insight generation) thus we generate a series of $2^3 = 8$ prompts and record the effect of the features both independently and in interactions to (i) find the best prompt and (ii) assess which features contribute to prompt effectiveness. Consequently, the prompts include dynamic elements (shown as `«element»`) that can be present or absent depending on whether the corresponding binary variable was turned on or off in the experimental design. The prompts are also populated based on the extracted data, shown as *[data]*.

Prompt 1 sets the task. The only prompt element changed by the experimental design is the *role*, also known as 'persona' [74, 83]. By telling the LLM that it is an expert in a specific domain, we guides its tone, style, and type of response. We noticed that LLMs tended to go beyond the task of providing a context as they included summaries or conclusions, which were not only unnecessary but also occasionally erroneous since they did not account for simulation results. To prevent LLMs from making broad generalizations just based on the goal of a model and its parameters, we explicitly request *not* to write any summary or conclusion.

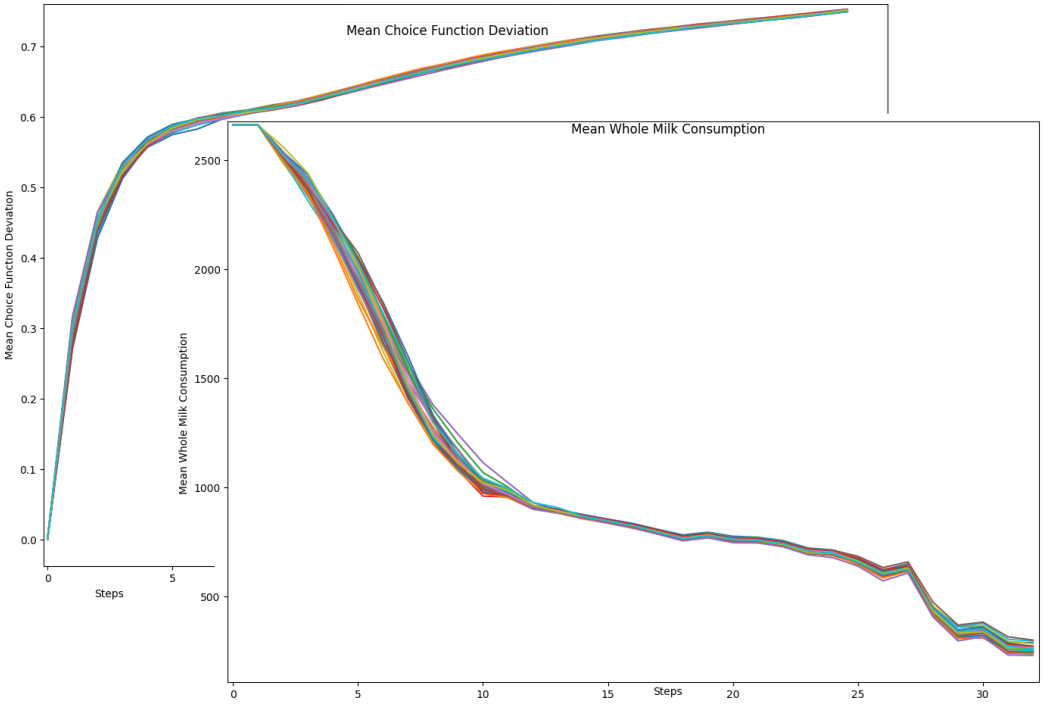

Figure 5: Two plots automatically created from CSV files. They are stacked for a concise presentation in this article only. Note that the plots do not necessarily have a monotonic increase or decrease, thus their explanation in textual form and the ensuing summaries have to identify which inflections are worth discussing. These examples are from the Milk Adoption ABM, used in our experiments (Section 4.1).

Prompt 2 describes the data from the simulation, one variable at a time, which is the appropriate place to make conclusions based on evidence. Simulation results were extracted as structured data (see section 3.2) which leaves three options to pass them to the LLM: (i) provide the raw CSV files; (ii) provide statistical summaries; (iii) provide visuals as time series. While LLMs can interpret results and generate insights, they are not designed for statistically rigorous analyses of numerical accuracy at scale, as a large amount of data may exceed their context size [84] and researchers continue to report issues in direct process of structured data [85]. We thus opted against a direct upload of simulation outputs. Uploading time series images to an LLM can quickly extract high-level insights, especially when trends and anomalies are visually clear. While not as precise as statistical tools, the vision ability of LLMs can identify patterns and generating hypotheses efficiently, which

addresses the need to communicate findings in an executive report. We thus chose the third option by automatically generating plots from the simulation data and providing it along with the prompt (Figure 5). In addition to the role, this prompt has two dynamic elements. The *example* illustrates the style of an executive report for simulation models; a sample is provided in our supplementary material (page 8, "Here is an example of the style of the report:"). The *insight* adds "When summarizing trends, provide brief insights about their implications for decision makers." In principles, a summary tells the reader what is there while insights explain why it matters. However, requesting insights might lead the LLM to hallucinate by seeing causality where the data does not support it (since LLMs are based on patterns in data rather than genuine comprehension [86]), or their tendency to prioritize interesting find may give the summary a suddenly dramatic flair[1]

Section S2 of our supplementary material provides a sample instance for both prompts, illustrating how the dynamic elements are replaced by specific instructions.

Prompt 1: Generate context

`«Role»` Your goal is to explain an agent-based model. Your explanation must include the context of the model, its goals, and key parameters. For each parameter, include the range of values (if provided) and the value that we have set. Do not write any summary or conclusion. *[Retrieve parameters from model.]* The context and goals of the model are as follows. *[Embed static context of the model here.]*

[1]The literature has demonstrated the potential of generating insights via LLM when the data consists of a single table, the goals are clear, and the data is clean – as is the case in our work. If the LLM is intended to navigate a database (e.g., multi-hop reasoning that joins databases), goals are unknown, and the data needs fixing (e.g., missing values), then additional processing is needed. For example, a hypothesis generator can create potential insights [87], which are tested against the data by a query agent [88].

Prompt 2

«Role» We have a plot from repeated simulations of an agent based model. Your goal is to describe the trends in details from the plot, mentioning key time steps and values taken by the metric in the plot, and interpreting them based on the context of the model. The report must objectively describe the trends in the data without addressing the quality of the simulation. Do not refer to the plot or any visual in your description. If a plot has very simple dynamics, simply state them without expanding. «Example» «Insights» *[The attachment is the plot representing the average weekly consumption of whole milk per agent]*

3.3.2. Post-processing

In line with best practices, we ask the LLM to first generate a full response (i.e., without constraints on lengths), then we summarize it. The responses can be particularly long, since each variable is depicted as a visual that gets transformed into text. For example, if an ABM has 15 variables, then we create 15 CSV files (each comprised of several runs), leading to 15 complete descriptions through Prompt 2. Long document summarization is its own challenge and has previously been addressed by a hierarchical approach, where each segment is summarized (e.g., 15 summaries) and then pooled to create one summary [89]. We follow this two-step approach by summarizing the description of each plot, which are further summarized into a final report consisting of one paragraph for the context and three paragraphs for the plots. We considered both extractive (BERT) and abstractive (BART) techniques to generate the summaries (subsection 2.3). Because of its extractive approach, BERT may combine 'fragments' to form sentences. This required further automated corrections including punctuation, spaces in parentheses, hyphens, or setting the first letter of each sentence to upper case.

3.4. Evaluation

For each simulation model, a summary was jointly written by the authors to serve as ground truth reference. We wrote the summary given the same information as was available to the LLMs, including the time series from each simulation run. All summaries were written prior to generating text via LLMs, thus ensuring that the ground truth reference provided by our

summary was not 'contaminated' by sentences that we may read in the LLM output. The references are used to compute five scores (subsection 2.3): BLEU [90, 65, 91], METEOR [91], ROUGE-1, ROUGE-2, ROUGE-L [90, 66, 91]. In addition to the five reference-based scores, we use a reference-free evaluation that computes the readability of the report. This is evaluated using the Flesch Readability Score [71].

4. Results

4.1. Experimental setup

We considered 3 LLMs and 3 ABMs. We selected GPT-4o and Claude 3.5 Sonnet as they were the leading solutions for multimodal learning at the time when the project was designed (`chat.lmsys.org/?leaderboard`, rank on July 1st 2024). We also included DeepSeek R1, as this LLM received significant attention following its release in 2025. The three ABMs were sourced from recent peer-reviewed publications created by different research groups and covering various topics, in order to evaluate the ability of our approach across application domains. The ABMs had to be adequately documented in order to provide sufficient context for our LLMs to operate. Given these criteria, we used three ABMs from the ComSES.net repository:

- The Megafaunal Hunting Pressure Model (MHPM) by Ullah and colleagues (Figure 4), released in late 2024 to "understand how environmental dynamics and human predation preferences interact with ungulate life history characteristics to affect ungulate population dynamics over time" [92].

- The Milk Adoption ABM from Gibson *et al.* [93], designed to evaluate different mechanisms of consumer decision-making in milk choice substitution to better understand and replicate historical trends in British milk consumption as a step toward modelling sustainable dietary shifts.

- The extended RAGE model [94], which explores how different types of learning (learning-by-doing and social learning) and varying attitudes toward risk influence smallholder farmers' decisions about livestock management and the resulting outcomes for their livelihoods and the environment.

For replicability, the libraries used in each part are summarized in Table 4.1. Given our design, we have the following number of experiments:

$$\underbrace{2^3}_{\text{factorial design with 3 binary prompt elements (role, insight, example)}} \times \underbrace{3}_{\text{simulation repeats}}$$
$$\times \underbrace{2}_{\text{summarization algorithms}} \times \underbrace{3}_{\text{Agent-Based Models}} \times \underbrace{3}_{\text{LLMs}}$$
$$= 432 \text{ reports generated and evaluated with 5 quantitative metrics.}$$

Table 1: Python Libraries used including their role in our method

Parts	**Use**	**Python Library**
Data Extraction	Running the simulation	pynetlogo (0.5.2)
	Extracting the static data	re (2.2.1), json (2.0.9)
Report Generation	Plotting the data	pandas (1.5.3), matplotlib (3.8.3)
	Generating explanations	openai (1.14.3), anthropic (0.29.0)
	Converting image to base64	base64 (python 3.10.9)
	Combining prompt features	itertools (python 3.10.9)
	Summarizing	transformers (4.41.2)
Evaluation	Scoring	nltk (3.7), rouge_score (0.1.2)

4.2. Quantitative Results

4.2.1. Optimal performances

The best results reported in Table 2 show that performances heavily depend on the experimental setting and on the preferred metric, as different LLMs optimize different aspects of the text and react differently across ABMs. Despite this volatility, we see that Claude is often the best choice (4 out of 6 times) for reading ease while GPT is often the best choice (4 out of 6 times) for most of the BLEU, ROUGE, and METEOR metrics. The Flesch readability ease rewards shorter words, shorter sentences, and simpler syntax, which are aligned with Claude's tendency to generate less formal structures in conversational tone. In contrast, GPT is heavily optimized

for downstream NLP benchmarks such as captured by BLEU, ROUGE, and METEOR, where being conservative in paraphrasing is an asset. DeepSeek is a surprisingly competitive solution although its strengths are less clear, as it does very well for the RAGE ABM (sometimes outperforming the other two LLMs by a wide margin) but is weaker on the other two ABMs. The intermediate positioning of DeepSeek was confirmed by our t-tests, as there is a statistically significant difference between DeepSeek and GPT on reading ease (p=0.0035, t=3.00) and also a statistically significant difference between DeepSeek and Claude on METEOR (p=0.0020, t=3.18).

When considering the summarization algorithms, BART produces summaries closer in phrasing and structure to the human-written summaries. This suggests that summaries written by humans are not copied from source documents but rather rewritten or paraphrased by modelers to provide a high-level picture. To contextualize the influence of the ABMs, note that MHPM is about ecological and evolutionary modeling, which may be more technical and abstract. This technical language and complexity make the summaries harder to simplify and they may favor LLMs that can reproduce domain-specific vocabulary. The Milk Adoption ABM is about behavioral modeling with a consumer focus, discussing decision-making, preferences, and historical trends. This more familiar content lets GPT match wording and Claude produce smooth, easy-to-read text. DeepSeek's proficiency in handling the RAGE model potentially arises from its capacity to process and summarize content characterized by high variability, agent heterogeneity, and adaptive behaviors.

4.2.2. Impact of Experimental Design Factors

Table 3 reports statistically significant effects of an experimental variable (including the three dynamic prompt elements) across the metrics. We observe that the content and structure of the underlying ABM has a major influence on text generation quality – likely due to differences in domain complexity, vocabulary, and narrative style. The summarization technique has a clear impact on text fidelity and linguistic quality, especially for deeper n-gram overlap and ease of reading. The choice of LLM has a consistent effect on output, especially regarding precision and clarity. In contrast, the dynamic prompt elements show little to no statistical significance across metrics *except for the reading ease*.

Unlike ANOVA, which only tests for statistical significance, the factorial analysis summarized in Table 4 quantifies the relative contribution of

Table 2: For each ABM, LLM, and summarization algorithms, we report the best result based on optimizing the prompt.

Experimental condition			**Best score across dynamic prompt elements**					
ABM	***Summary***	***LLM***	BLEU	METEOR	R-1	R-2	R-3	Reading ease
Milk	BART (abstract)	Claude	0.11	0.35	0.58	0.22	0.27	63.70
		GPT	0.13	0.36	0.64	0.27	0.28	54.83
		DeepSeek	0.09	0.34	0.59	0.22	0.25	61.87
	BERT (extract)	Claude	0.09	0.28	0.57	0.21	0.24	59.84
		GPT	0.10	0.29	0.59	0.22	0.24	59.03
		DeepSeek	0.06	0.26	0.56	0.20	0.24	59.33
RAGE	BART (abstract)	Claude	0.03	0.23	0.42	0.10	0.17	57.27
		GPT	0.03	0.26	0.44	0.11	0.18	55.44
		DeepSeek	0.04	0.30	0.50	0.17	0.20	56.25
	BERT (extract)	Claude	0.03	0.19	0.40	0.09	0.16	52.09
		GPT	0.02	0.20	0.42	0.09	0.17	51.99
		DeepSeek	0.04	0.25	0.47	0.12	0.21	60.04
MHPM	BART (abstract)	Claude	0.03	0.25	0.40	0.10	0.17	54.52
		GPT	0.04	0.25	0.41	0.10	0.17	47.69
		DeepSeek	0.02	0.27	0.33	0.07	0.15	45.46
	BERT (extract)	Claude	0.04	0.21	0.40	0.10	0.17	51.07
		GPT	0.04	0.21	0.42	0.10	0.17	51.18
		DeepSeek	0.03	0.25	0.40	0.08	0.17	51.18

each factor to total variance, also making it possible to compare their *joint influence* on model performance. That is, we show *dependencies in the architecture* as the same prompt yields different outcomes depending on the LLM or summarizer used. While these results confirm the role of the summarization algorithm and the LLM, they provide additional information to the ANOVA. The summarization algorithm (BART vs. BERT) determines how the ABM outputs are reworded and what information is emphasized, which impacts metrics that directly measure lexical overlap (BLEU, METEOR, and ROUGE scores). So the choice of summarization algorithm clearly dominates the variance in these metrics, especially for METEOR, which also accounts for paraphrasing and synonyms (where BART's generative style may shine more). As we know from the ANOVA, the LLM choice was statistically significant for BLEU and ROUGE-1, which are sensitive to surface-level lexical overlaps where LLMs clearly differ (e.g., Claude paraphrasing more, GPT

Table 3: p-value from a one-way ANOVA between our six experimental variables (including the three dynamic prompt elements in the bottom rows) and our six quantitative metrics, where 'R' denotes ROUGE.

↓*Variable / metric* →	BLEU	METEOR	R-1	R-2	R-L	Reading ease
Agent-Based Model	<0.01	<0.01	<0.01	<0.01	<0.01	<0.01
Summarization algorithm	<0.01	<0.01	0.09	<0.01	<0.01	0.71
LLM	<0.01	<0.01	<0.01	0.10	0.21	<0.01
Use of roles	0.83	0.89	0.96	0.35	0.42	0.36
Generating insights	0.45	0.30	0.37	0.73	0.65	<0.01
Providing examples	0.76	0.49	0.53	0.32	0.26	<0.01

being more literal). The LLM does not seem to affect METEOR because the summarization algorithm dominates. This suggests that the summarization algorithm tightly constrains the input to the LLM, thus limiting its influence to rephrased already-summarized content. *Most interestingly, the factorial analysis reveals that prompt elements do matter either individually or by interacting with the other technical choices.* Asking for insights has a noticeably impact on reading ease since it affects how the LLM frames the outputs by using explanatory language and potentially staying away from dense or technical phrasing. Interactions between insights and examples emerge because examples help ground the response while insights elevate reasoning, leading to a small improvement on n-gram matches (ROUGE-10 with reference summaries written in similar style. The interactions of LLMs and examples for ROUGE-1 and ROUGE-2 suggest that LLMs interpret example-driven prompts in different ways (e.g., Claude may elaborate more). The interactions of summarization algorithms and insights for BLEU potentially reflect that requests for insights generate more reasoning or causal language, which can be captured more fluidly in abstractive summaries than in extractive summaries.

Our supplementary material (section S3) provides the correlation matrix among the scores, which helps to verify expected relationships. For example, lexical overlap metrics tend to be positively correlated, while readability measures like Flesch behave differently. This is provided for completeness to document consistency rather than generate new insights into our architecture, which is the focus of this analysis.

Table 4: Effect of individual prompt elements and pairs of elements onto the quantitative scores. This uses a factorial design of experiments, thus the scores correspond to the percentage of contribution to variance. Scores above 5 are emphasized **in bold.**

Feature	BLEU	METEOR	R-1	R-2	R-L	Reading ease
Example	1.85	0.39	**5.09**	0.56	0.90	**19.32**
Example *and* Insight	0.07	0.56	**5.49**	2.05	2.02	0.10
Insight	0.30	0.17	2.52	4.63	**5.66**	**43.64**
LLM	**29.07**	2.70	**32.87**	3.06	0.26	**21.57**
LLM *and* Example	0.24	0.47	**9.98**	**6.94**	1.10	4.68
LLM *and* Insight	0.19	0.08	0.58	0.28	0.74	3.15
LLM *and* Role	1.95	0.08	0.13	0.81	0.69	1.94
Role	0.14	0.01	0.01	4.01	2.91	1.47
Role *and* Example	0.43	0.01	0.42	3.21	0.49	1.21
Role *and* Insight	1.15	0.11	1.21	0.46	0.07	1.86
Summary	**51.29**	**94.93**	**18.72**	**72.44**	**55.97**	0.25
Summary *and* Example	0.62	0.33	0.52	0.30	1.63	0.03
Summary *and* Insight	**6.93**	0.12	0.00	0.30	0.31	0.63
Summary *and* LLM	**5.27**	0.04	**20.97**	0.21	**26.32**	0.02
Summary *and* Role	0.50	0.02	1.50	0.75	0.93	0.14

5. Discussion

Our study automatically translated *well-documented* ABMs into executive reports, which were evaluated both quantitatively (as detailed in this article) and qualitative (as available in the supplementary online material). Our quantitative evaluations show that optimizing the configuration of LLMs, summarization algorithms, and prompt elements can create report whose readability oscillates between a college-educated audience or the general public (readability 50-70). From a quantitative standpoint, there is thus evidence that the generated overviews of the ABMs are accessible, which was a challenge given that the readability of conceptual models once turned into text was under 50 in a recent study [14]. We found that the more desirable reports were created through abstractive summarization, by taking a high-level view of a model and rephrasing its content, rather than by extractive summarization in which existing bits and pieces of a long report are reorganized. From a scoring viewpoint, abstractive summarization naturally results in lower overlap scores between a generated summary and a target, since the

creative paraphrasing reduces the amount of direct matches even when the meaning is preserved. Our scores on n-gram matches (BLEU, METEOR, ROUGE) are thus within expected ranges for abstractive summarization on complex, technical content like ABM outputs. Although we performed a complementary qualitative evaluation to be comprehensive, the low consistency of ratings between human raters or between human raters and LLMs on well-established metrics (coverage, faithfulness) reflects documented challenges in the modeling literature, such as the notable differences in parsing information from reports and relating it to a model's structure [95].

While the literature on prompt engineering and the optimization of LLM-based systems predominantly uses simple experimental designs (e.g., varying one factor at a time, or a limited 2^2 factorial design), we took a thorough approach to analyzing the influence of each design element and their contributions through a factorial analysis with replication ($2^k r$ design). Our analysis illustrates the benefits of using a well-established approach to designed experiments in the field of Generative AI, where it has not yet been fully utilized. Specific to our work, the analysis reveals that the choice of summarization algorithms account for most of the output variance on n-gram scores, which can be surprising given that the emphasis is often on the LLMs or the prompt optimization. The prompt elements affect readability and interact with other controllable aspects of the pipeline, as some LLMs are able to leverage examples in the prompt. Our evaluation showed that three state-of-the-art LLMs (DeepSeek, GPT, Claude) have different benefits, as Claude's natural-sounding style favors readability, GPT's more formal descriptions tend to align with target summaries, and DeepSeek occasionally provide unique performances.

The main limitation of our approach is that the ABMs must be accompanied by documentation that provides enough guidance to the generative models. Solidly anchoring a summary in facts can only be achieved if the task is strictly about translating rather than story-telling, which can hallucinate. The simulation community regularly stresses the importance of documenting the models [35] and repositories such as CoMSES provide a wealth of well-documented ABMs. However, there is no guarantee that *any* ABM comes with sufficient documentation, even if it is intended for use by a broad set of stakeholders. An important objective for future works would be to identify documentations for models that are insufficiently described. This may be achieved in at least two ways. First, clone detectors can realize that a model (or parts thereof) resembles other, well-known models thus some of

the documentation may be reused. The challenge would be to only reuse the applicable parts of the documentation. Second, empirical studies on ABMs have shown that related models (e.g., ABMs for social simulations) often shared 'building blocks' [96], for instance to simulate human mobility. Automatically identifying these building blocks within a simulation model could thus generate some of the missing documentation.

While the integration of LLMs with simulation outputs is promising, our study also raises two questions that can be investigated in future works. First, our work used the documentation of the ABM as guidance for generative AI. However, a model may be described in a research article and/or it may use the specialized vocabulary of its application domain. The use of retrieval-augmented generation (RAG) could organize this knowledge in a database, thus providing supplemental guidance for generative AI. However, additional studies are needed to identify which aspects to extract and store in a database, or how to prioritize the database compared to the model's documentation or the LLM's own knowledge. Second, our analysis showed that the optimal LLM depends on the ABM, for example as DeepSeek generates summaries with the highest readability for one model while Claude is better for the other two models. It would thus be valuable to identify *which features* of a ABM are best aligned with the capability of a LLM. Automatically supporting this alignment instead of performing a complete optimization study each time would allow the audience of simulation models to quickly receive the best report for the model of their choice.

6. Conclusion

By automatically translating well-documented Agent-Based Models into concise executive reports, we lay the groundwork for integrating LLMs into simulation workflows to enhance transparency and stakeholder engagement. Future work can explore expanding this approach and reuse our open-access code for other simulation paradigms, or assess it across a broader range of application domains. While our quantitative metrics show that the reports have a sufficient level of readability, empirical studies with stakeholders can reveal whether the reports can bridge the gap between modelers and decision-makers depending on the model and the readership.

7. Glossary

ABM: Agent-Based Modeling, a discrete simulation technique where individual entities (agents) interact with each other and with their environment.

LLM: Large-Language Model (e.g., DeepSeek, GPT), a deep neural network that can generate text based on inputs such as text or on multiple modalities such as text and images.

NetLogo: A simulation environment for agent-based modeling.

References

[1] M. L. Zellner, D. Milz, L. Lyons, C. Hoch, J. Radinsky, Finding the balance between simplicity and realism in participatory modeling for environmental planning, Environmental Modelling & Software 157 (2022) 105481.

[2] G. Grigoryan, A. J. Collins, Feature importance for uncertainty quantification in agent-based modeling, in: 2023 Winter Simulation Conference (WSC), IEEE, 2023, pp. 233–242.

[3] M. S. Jalali, E. Beaulieu, Strengthening a weak link: Transparency of causal loop diagrams—current state and recommendations, System dynamics review 40 (4) (2024) e1753.

[4] S. Allender, B. Owen, J. Kuhlberg, J. Lowe, P. Nagorcka-Smith, J. Whelan, C. Bell, A community based systems diagram of obesity causes, PloS one 10 (7) (2015) e0129683.

[5] C. Siokou, R. Morgan, A. Shiell, Group model building: a participatory approach to understanding and acting on systems, Public Health Res Pract 25 (1) (2014) e2511404.

[6] A. Jack, Foresight report on obesity–author's reply, The Lancet 370 (9601) (2007) 1755.

[7] B. L. van Veen, J. Roland Ortt, Simplification errors in predictive models, Futures & Foresight Science 6 (3) (2024) e184.

[8] S. Swarup, Adequacy: what makes a simulation good enough?, in: 2019 Spring Simulation Conference (SpringSim), IEEE, 2019, pp. 1–12.

[9] J. F. Uleman, K. Stronks, H. Rutter, O. A. Arah, N. H. Rod, Mapping complex public health problems with causal loop diagrams, International Journal of Epidemiology 53 (4) (2024) dyae091.

[10] A. Shrestha, K. Mielke, T. A. Nguyen, P. J. Giabbanelli, Automatically explaining a model: Using deep neural networks to generate text from causal maps, in: 2022 Winter simulation conference (WSC), IEEE, 2022, pp. 2629–2640.

[11] A. Phatak, V. K. Mago, A. Agrawal, A. Inbasekaran, P. J. Giabbanelli, Narrating causal graphs with large language models, in: Proceedings of the Hawaii International Conference on System Sciences 2024 (HICSS-57), 2024.

[12] B. Hedelin, S. Gray, S. Woehlke, T. K. BenDor, A. Singer, R. Jordan, M. Zellner, P. Giabbanelli, P. Glynn, K. Jenni, et al., What's left before participatory modeling can fully support real-world environmental planning processes: A case study review, Environmental Modelling & Software 143 (2021) 105073.

[13] Y. Wei, P. Knoeferle, Causal inference: relating language to event representations and events in the world, Frontiers in psychology 14 (2023) 1172928.

[14] T. J. Gandee, P. J. Giabbanelli, Combining natural language generation and graph algorithms to explain causal maps through meaningful paragraphs, in: International Conference on Conceptual Modeling, Springer, 2024, pp. 359–376.

[15] P. J. Giabbanelli, Gpt-based models meet simulation: how to efficiently use large-scale pre-trained language models across simulation tasks, in: 2023 Winter simulation conference (WSC), IEEE, 2023, pp. 2920–2931.

[16] W. Schoenberg, D. Girard, S. Chung, E. O'Neill, J. Velasquez, S. Metcalf, How well can ai build sd models?, arXiv preprint arXiv:2503.15580 (2025).

[17] N. Hosseinichimeh, A. Majumdar, R. Williams, N. Ghaffarzadegan, From text to map: a system dynamics bot for constructing causal loop diagrams, System Dynamics Review 40 (3) (2024) e1782.

[18] G. Veldhuis, F. van Wermeskerken, O. Visker, P. Steinmann, S. Deuten, R. van Waas, From data to model structure: A generative algorithm to develop system dynamics models, SSRN 4921086.

[19] P. J. Giabbanelli, T. J. Gandee, A. Agrawal, N. Hosseinichimeh, Benchmarking and assessing transformations between text and causal maps via large language models, Applied Ontology (2025) 15705838241304102.

[20] J. C. Verduzco, E. Holbrook, A. Strachan, Gpt-4 as an interface between researchers and computational software: improving usability and reproducibility, arXiv preprint arXiv:2310.11458 (2023).

[21] G. Méndez, P. Gervás, Using chatgpt for story sifting in narrative generation, in: Proceedings of The 14th International Conference on Computational Creativity, 2023.

[22] N. Aoki, N. Mori, M. OKada, Analysis of llm-based narrative generation using the agent-based simulation, in: 2023 15th International Congress on Advanced Applied Informatics Winter (IIAI-AAI-Winter), IEEE, 2023, pp. 284–289.

[23] S. Achter, M. Borit, C. Cottineau, M. Meyer, J. G. Polhill, V. Radchuk, How to conduct more systematic reviews of agent-based models and foster theory development-taking stock and looking ahead, Environmental Modelling & Software 173 (2024) 105867.

[24] D. L. DeAngelis, S. G. Diaz, Decision-making in agent-based modeling: A current review and future prospectus, Frontiers in Ecology and Evolution 6 (2019) 237.

[25] F. Bianchi, F. Squazzoni, Agent-based models in sociology, Wiley Interdisciplinary Reviews: Computational Statistics 7 (4) (2015) 284–306.

[26] L. Qiu, R. Phang, Agent-based modeling in political decision making (02 2020).

[27] J.-S. Lee, T. Filatova, A. Ligmann-Zielinska, B. Hassani-Mahmooei, F. Stonedahl, I. Lorscheid, A. Voinov, J. G. Polhill, Z. Sun, D. C. Parker, The complexities of agent-based modeling output analysis, Journal of Artificial Societies and Social Simulation 18 (4) (2015).

[28] L. An, V. Grimm, B. L. Turner II, Meeting grand challenges in agent-based models, Journal of Artificial Societies and Social Simulation 23 (1) (2020).

[29] L. Papadopoulou, K. McEntaggart, J. Etienne, Communicating scientific uncertainty in advice provision to decision-makers: Review of approaches and recommendations for uk statutory nature conservation bodies, Joint Nature Conservation Committee (JNCC) Report 617 (2018).

[30] Z. Sun, I. Lorscheid, J. D. Millington, S. Lauf, N. R. Magliocca, J. Groeneveld, S. Balbi, H. Nolzen, B. Müller, J. Schulze, et al., Simple or complicated agent-based models? a complicated issue, Environmental Modelling & Software 86 (2016) 56–67.

[31] V. Grimm, S. F. Railsback, C. E. Vincenot, U. Berger, C. Gallagher, D. L. DeAngelis, B. Edmonds, J. Ge, J. Giske, J. Groeneveld, et al., The odd protocol for describing agent-based and other simulation models: A second update to improve clarity, replication, and structural realism, Journal of Artificial Societies and Social Simulation 23 (2) (2020).

[32] B. Müller, F. Bohn, G. Dreßler, J. Groeneveld, C. Klassert, R. Martin, M. Schlüter, J. Schulze, H. Weise, N. Schwarz, Describing human decisions in agent-based models–odd+ d, an extension of the odd protocol, Environmental Modelling & Software 48 (2013) 37–48.

[33] A. Laatabi, N. Marilleau, T. Nguyen-Huu, H. Hbid, M. Ait Babram, Odd+ 2d: An odd based protocol for mapping data to empirical abms, Journal of Artificial Societies and Social Simulation 21 (2) (2018).

[34] S. Achter, M. Borit, E. Chattoe-Brown, P.-O. Siebers, Rat-rs: a reporting standard for improving the documentation of data use in agent-based modelling, International journal of social research methodology 25 (4) (2022) 517–540.

[35] T. Monks, C. S. Currie, B. S. Onggo, S. Robinson, M. Kunc, S. J. Taylor, Strengthening the reporting of empirical simulation studies: Introducing the stress guidelines, Journal of Simulation 13 (1) (2019) 55–67.

[36] P. J. Giabbanelli, B. Tison, J. Keith, The application of modeling and simulation to public health: Assessing the quality of agent-based models for obesity, Simulation Modelling Practice and Theory 108 (2021) 102268.

[37] A. Hall, K. Virrantaus, Visualizing the workings of agent-based models: Diagrams as a tool for communication and knowledge acquisition, Computers, Environment and Urban Systems 58 (2016) 1–11.

[38] L. Szangolies, M.-S. Rohwäder, H. Ahmed, F. Jahanmiri, A. Wagner, R. Souto-Veiga, V. Grimm, C. Gallagher, Visual odd: A standardised visualisation illustrating the narrative of agent-based models, Journal of Artificial Societies and Social Simulation 27 (4) (2024).

[39] A. Dorin, N. Geard, The practice of agent-based model visualization, Artificial life 20 (2) (2014) 271–289.

[40] D. Kornhauser, U. Wilensky, W. Rand, Design guidelines for agent based model visualization, Journal of Artificial Societies and Social Simulation 12 (2) (2009) 1.

[41] A. Vandin, D. Giachini, F. Lamperti, F. Chiaromonte, Automated and distributed statistical analysis of economic agent-based models, Journal of Economic Dynamics and Control 143 (2022) 104458.

[42] M. Hooten, C. Wikle, M. Schwob, Statistical implementations of agent-based demographic models, International Statistical Review 88 (2) (2020) 441–461.

[43] Ö. Gürcan, Llm-augmented agent-based modelling for social simulations: Challenges and opportunities, HHAI 2024: Hybrid Human AI Systems for the Social Good (2024) 134–144.

[44] A. Ferraro, A. Galli, V. La Gatta, M. Postiglione, G. M. Orlando, D. Russo, G. Riccio, A. Romano, V. Moscato, Agent-based modelling

meets generative ai in social network simulations, in: International Conference on Advances in Social Networks Analysis and Mining, Springer, 2024, pp. 155–170.

[45] T. Hu, D. Liakopoulos, X. Wei, R. Marculescu, N. J. Yadwadkar, Simulating rumor spreading in social networks using llm agents, arXiv preprint arXiv:2502.01450 (2025).

[46] N. Ghaffarzadegan, A. Majumdar, R. Williams, N. Hosseinichimeh, Generative agent-based modeling: an introduction and tutorial, System Dynamics Review 40 (1) (2024) e1761.

[47] M. Larooij, P. Törnberg, Do large language models solve the problems of agent-based modeling? a critical review of generative social simulations, arXiv preprint arXiv:2504.03274 (2025).

[48] D. B. Acharya, K. Kuppan, B. Divya, Agentic ai: Autonomous intelligence for complex goals–a comprehensive survey, IEEE Access (2025).

[49] J. J. Padilla, D. Shuttleworth, K. O'Brien, Agent-based model characterization using natural language processing, in: 2019 Winter Simulation Conference (WSC), IEEE, 2019, pp. 560–571.

[50] D. Shuttleworth, J. J. Padilla, Towards semi-automatic model specification, in: 2021 Winter Simulation Conference (WSC), IEEE, 2021, pp. 1–12.

[51] S. Khatami, Ai-enhanced abm development: Facilitating agent-based modeling using artificial intelligence, Phd thesis, Norwegian University of Science and Technology (2025).

[52] C. W. Davis, A. J. Jetter, P. J. Giabbanelli, Towards an automatic construction of simulation scenarios: A systematic review, in: 2023 Winter Simulation Conference (WSC), IEEE, 2023, pp. 2494–2505.

[53] J. Martínez, B. Llinas, J. G. Botello, J. J. Padilla, E. Frydenlund, Enhancing gpt-3.5's proficiency in netlogo through few-shot prompting and retrieval-augmented generation, in: 2024 Winter Simulation Conference (WSC), IEEE, 2024, pp. 666–677.

[54] E. Frydenlund, J. Martínez, J. J. Padilla, K. Palacio, D. Shuttleworth, Modeler in a box: how can large language models aid in the simulation modeling process?, Simulation 100 (7) (2024) 727–749.

[55] A. Akhavan, M. S. Jalali, Generative ai and simulation modeling: how should you (not) use large language models like chatgpt, System Dynamics Review 40 (3) (2024) e1773.

[56] I. Jackson, B. Rolf, Do natural language processing models understand simulations? application of gpt-3 to translate simulation source code to english, IFAC-PapersOnLine 56 (2) (2023) 221–226.

[57] D. Miller, Leveraging bert for extractive text summarization on lectures, arXiv preprint arXiv:1906.04165 (2019).

[58] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, L. Zettlemoyer, Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, arXiv preprint arXiv:1910.13461 (2019).

[59] T. August, K. Lo, N. A. Smith, K. Reinecke, Know your audience: The benefits and pitfalls of generating plain language summaries beyond the" general" audience, in: Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems, 2024, pp. 1–26.

[60] National Academies of Sciences, Engineering, and Medicine, Executive summary, in: Modeling Human and Organizational Behavior: Application to Military Simulations, The National Academies Press, Washington, DC, 1998. doi:10.17226/6173.
URL https://doi.org/10.17226/6173

[61] National Academies of Sciences, Engineering, and Medicine, Executive summary, in: Behavioral Modeling and Simulation: From Individuals to Societies, The National Academies Press, Washington, DC, 2008. doi:10.17226/12169.
URL https://doi.org/10.17226/12169

[62] V. Kho, Mckinsey executive summaries: Short but powerful, Writing for University and Beyond: A Journal of First-Year Student Writing at UTM 2 (1) (2022) 87–95.

[63] L. N. Brodsky (2018). [link].
URL https://projects.iq.harvard.edu/files/hks-communications-program/files/how_to_write_an_exex_summ_to_use_4_18_18.pdf

[64] T. A. van Schaik, B. Pugh, A field guide to automatic evaluation of llm-generated summaries, in: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2024, pp. 2832–2836.

[65] K. Papineni, S. Roukos, T. Ward, W.-J. Zhu, Bleu: a method for automatic evaluation of machine translation, in: Proceedings of the 40th annual meeting of the Association for Computational Linguistics, 2002, pp. 311–318.

[66] L. Chin-Yew, Rouge: A package for automatic evaluation of summaries, in: Proceedings of the Workshop on Text Summarization Branches Out, 2004, 2004, pp. 74–81.

[67] S. D. Myla, E. R. Saini, E. N. Kapoor, Auto text summarization in natural language processing, in: 2024 2nd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT), IEEE, 2024, pp. 1258–1267.

[68] C. Caparrós-Laiz, J. A. García-Díaz, R. Valencia-García, Evaluating extractive automatic text summarization techniques in spanish, in: International Conference on Technologies and Innovation, Springer, 2021, pp. 79–92.

[69] S. S. Roy, R. E. Mercer, Generating extractive and abstractive summaries in parallel from scientific articles incorporating citing statements, in: Proceedings of the 4th New Frontiers in Summarization Workshop, 2023, pp. 75–86.

[70] Y. Guo, W. Qiu, Y. Wang, T. Cohen, Automated lay language summarization of biomedical scientific reviews, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35, 2021, pp. 160–168.

[71] S. R. Cooperman, R. A. Brandão, Investigating the proficiency of an ai tool in summarizing foot and ankle literature: A quantitative, qualita-

tive and accuracy analysis, Foot & Ankle Surgery: Techniques, Reports & Cases 4 (2) (2024) 100384.

[72] Z. Wang, G. Xu, M. Ren, Llm-generated natural language meets scaling laws: New explorations and data augmentation methods, arXiv preprint arXiv:2407.00322 (2024).

[73] B. Chen, Z. Zhang, N. Langrené, S. Zhu, Unleashing the potential of prompt engineering in large language models: a comprehensive review, arXiv preprint arXiv:2310.14735 (2023).

[74] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, D. C. Schmidt, A prompt pattern catalog to enhance prompt engineering with chatgpt, arXiv preprint arXiv:2302.11382 (2023).

[75] M. Braun, M. Greve, F. Kegel, L. M. Kolbe, P. E. Beyer, Can (a)i have a word with you? a taxonomy on the design dimensions of ai prompts, in: 57th Annual Hawaii International Conference on System Sciences, HICSS 2024, Hawaii International Conference on System Sciences (HICSS), 2024, pp. 559–568.

[76] L. Memmert, I. Cvetkovic, E. Bittner, The more is not the merrier: Effects of prompt engineering on the quality of ideas generated by gpt-3, in: Proceedings of the 57th Hawaii International Conference on System Sciences, 2024.

[77] R. Amini, S. S. Norouzi, P. Hitzler, R. Amini, Towards complex ontology alignment using large language models, in: International Knowledge Graph and Semantic Web Conference, Springer, 2024, pp. 17–31.

[78] A. Gutschmidt, B. Nast, Assessing model quality using large language models, in: IFIP Working Conference on The Practice of Enterprise Modeling, Springer, 2024, pp. 105–122.

[79] M. J. Saeedizade, E. Blomqvist, Navigating ontology development with large language models, in: European Semantic Web Conference, Springer, 2024, pp. 143–161.

[80] Y. Zhao, N. Vetter, K. Aryan, Using large language models for ontoclean-based ontology refinement, arXiv preprint arXiv:2403.15864 (2024).

[81] S. M. Sanchez, P. J. Sanchez, H. Wan, Work smarter, not harder: A tutorial on designing and conducting simulation experiments, in: 2020 Winter Simulation Conference (WSC), IEEE, 2020, pp. 1128–1142.

[82] S. M. Sanchez, Robust design: Seeking the best of all possible worlds, in: 2000 winter simulation conference proceedings (cat. no. 00ch37165), Vol. 1, IEEE, 2000, pp. 69–76.

[83] M. Stahl, L. Biermann, A. Nehring, H. Wachsmuth, Exploring llm prompting strategies for joint essay scoring and feedback generation, arXiv preprint arXiv:2404.15845 (2024).

[84] S. Pawar, S. Tonmoy, S. Zaman, V. Jain, A. Chadha, A. Das, The what, why, and how of context length extension techniques in large language models–a detailed survey, arXiv preprint arXiv:2401.07872 (2024).

[85] P. Li, Y. He, D. Yashar, W. Cui, S. Ge, H. Zhang, D. Rifinski Fainman, D. Zhang, S. Chaudhuri, Table-gpt: Table fine-tuned gpt for diverse table tasks, Proceedings of the ACM on Management of Data 2 (3) (2024) 1–28.

[86] C. Cuskley, R. Woods, M. Flaherty, The limitations of large language models for understanding human language and cognition, Open Mind 8 (2024) 1058–1083.

[87] A. Abaskohi, A. V. Ramesh, S. Nanisetty, C. Goel, D. Vazquez, C. Pal, S. Gella, G. Carenini, I. H. Laradji, Agentada: Skill-adaptive data analytics for tailored insight discovery, arXiv preprint arXiv:2504.07421 (2025).

[88] A. S. Pérez, A. Boukhary, P. Papotti, L. C. Lozano, A. Elwood, An llm-based approach for insight generation in data analysis, arXiv preprint arXiv:2503.11664 (2025).

[89] Y.-J. Yin, B.-Y. Chen, B. Chen, A novel llm-based two-stage summarization approach for long dialogues, in: 2024 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC), IEEE, 2024, pp. 1–6.

[90] X. Zhang, Y. Li, J. Wang, B. Sun, W. Ma, P. Sun, M. Zhang, Large language models as evaluators for recommendation explanations, in: Proceedings of the 18th ACM Conference on Recommender Systems, 2024, pp. 33–42.

[91] N. B. Brown, Enhancing trust in llms: Algorithms for comparing and interpreting llms, arXiv preprint arXiv:2406.01943 (2024).

[92] M. C. Kopels, I. I. Ullah, Modeling post-pleistocene megafauna extinctions as complex social-ecological systems, Quaternary Research 121 (2024) 1–14.

[93] M. Gibson, J. P. Pereira, R. Slade, J. Rogelj, Agent-based modelling of future dairy and plant-based milk consumption for uk climate targets, Journal of Artificial Societies and Social Simulation 25 (2) (2022).

[94] G. Dressler, J. Groeneveld, C. M. Buchmann, C. Guo, N. Hase, J. Thober, K. Frank, B. Müller, Implications of behavioral change for the resilience of pastoral systems—lessons from an agent-based model, Ecological Complexity 40 (2019) 100710.

[95] A. J. Freund, P. J. Giabbanelli, Are we modeling the evidence or our own biases? a comparison of conceptual models created from reports, in: 2021 annual modeling and simulation conference (ANNSIM), IEEE, 2021, pp. 1–12.

[96] S. Cheng, P. J. Giabbanelli, Z. Kuang, Identifying the building blocks of social simulation models: A qualitative analysis using open-source codes in netlogo, in: 2023 Annual Modeling and Simulation Conference (ANNSIM), IEEE, 2023, pp. 306–317.